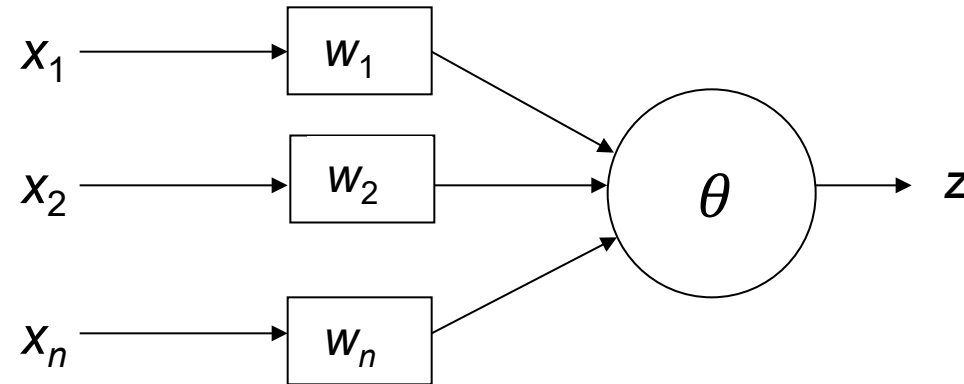
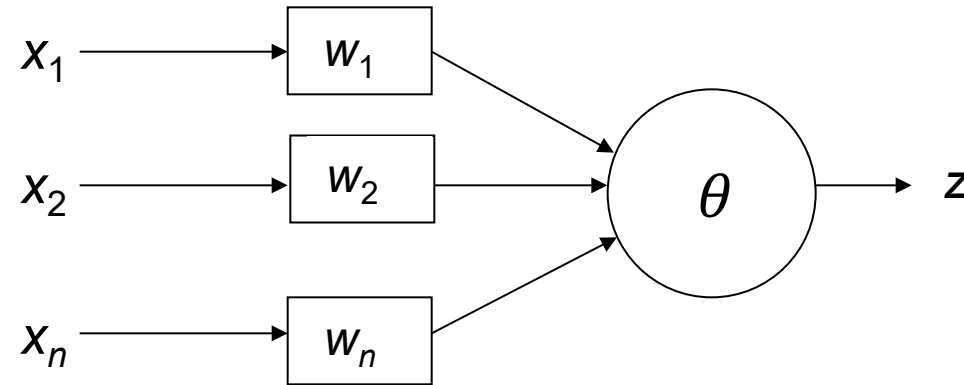


A Simple Perceptron Node – Threshold Logic Unit



$$z = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i w_i \geq \theta \\ 0 & \text{if } \sum_{i=1}^n x_i w_i < \theta \end{cases}$$

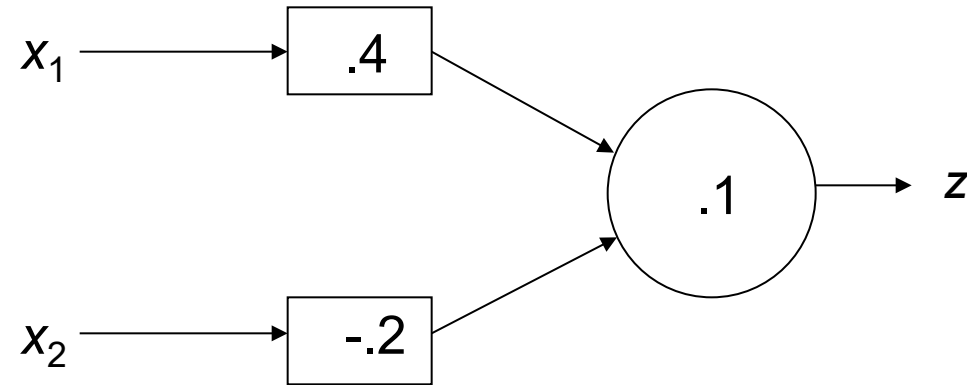
A Simple Perceptron Node – Threshold Logic Unit



- ❑ Learn weights such that an objective function is maximized.
- ❑ What objective function should we use?
- ❑ What learning algorithm should we use?

$$z = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i w_i \geq \theta \\ 0 & \text{if } \sum_{i=1}^n x_i w_i < \theta \end{cases}$$

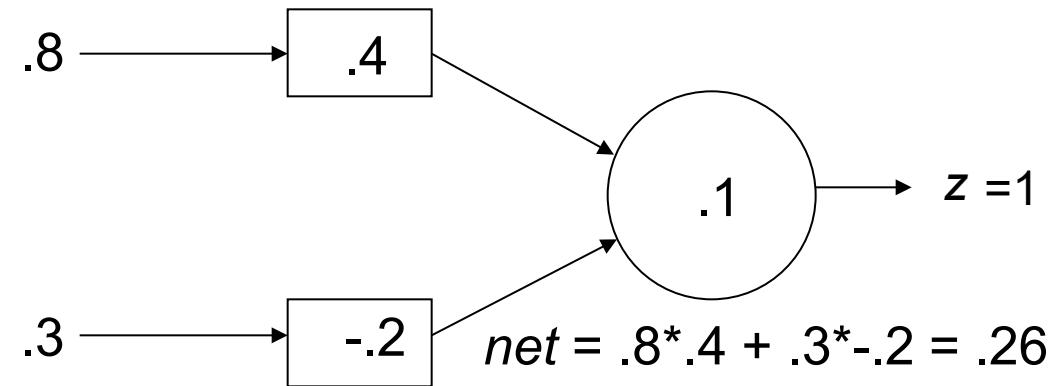
Perceptron Learning Algorithm



x_1	x_2	t
.8	.3	1
.4	.1	0

$$z = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i w_i \geq \theta \\ 0 & \text{if } \sum_{i=1}^n x_i w_i < \theta \end{cases}$$

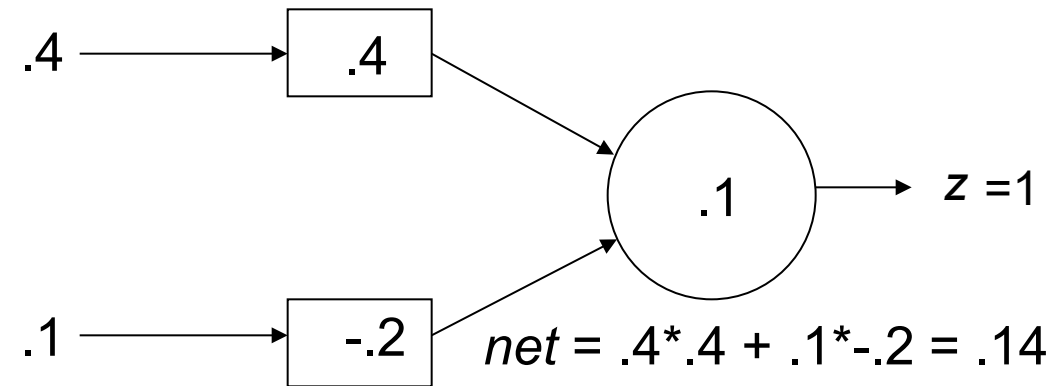
First Training Instance



x_1	x_2	t
.8	.3	1
.4	.1	0

$$z = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i w_i \geq \theta \\ 0 & \text{if } \sum_{i=1}^n x_i w_i < \theta \end{cases}$$

Second Training Instance



x_1	x_2	t
.8	.3	1
.4	.1	0

$$z = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i w_i \geq \theta \\ 0 & \text{if } \sum_{i=1}^n x_i w_i < \theta \end{cases}$$

$$\Delta w_i = c(t - z) x_i$$

Perceptron Rule Learning

$$\Delta w_i = c(t - z) x_i$$

- ❑ Where w_i is the weight from input i to perceptron node, c is the learning rate, t is the target for the current instance, z is the current output, and x_i is i^{th} input
- ❑ Least perturbation principle
 - ❑ Only change weights if there is an error
 - ❑ small c rather than changing weights sufficient to make current pattern correct
 - ❑ Scale by x_i

Perceptron Learning Rule (Cont.)

- ❑ Create a perceptron node with n inputs
- ❑ Iteratively apply a pattern from the training set and apply the perceptron rule
- ❑ Each iteration through the training set is an *epoch*
- ❑ Continue training until total training set error ceases to improve
- ❑ Perceptron Convergence Theorem: Guaranteed to find a solution in finite time if a solution exists